

Started on	Tuesday, 10 June 2025, 4:15 PM
State	Finished
Completed on	Thursday, 12 June 2025, 1:01 AM
Time taken	1 day 8 hours
Marks	4.00/4.00
Grade	10.00 out of 10.00 (100%)



Viết chương trình đọc một đồ thị **vô hướng** từ bàn phím và đếm số bộ phận liên thông của đồ thị.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v) .

Đầu ra (Output)

- In ra số bộ phận liên thông của đồ thị.

Gợi ý

- Xem tài liệu thực hành

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

For example:

Input	Result
4 3 2 1 1 3 2 4	1

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
```

```

31 //-----DFS-----
32
33 int cnt;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     visited[s] = 1;
37     // printf("%d \n", s);
38     for (int u = 1; u <= pG->n; u++) {
39         if (pG->A[s][u] > 0 && !visited[u]) {
40             DFS(pG, u);
41         }
42     }
43 }
44
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     // DFS(&G, 1);
62
63     cnt = 0;
64     for (int i = 1; i <= G.n; i++) {
65         if (!visited[i]) {
66             cnt++;
67             DFS(&G, i);
68         }
69     }
70     printf("%d", cnt);
71 }
72

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 2 4	1	1	✓
✓	4 2 1 2 3 4	2	2	✓
✓	4 2 1 4 2 3	2	2	✓

	Input	Expected	Got	
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5 6	1	1	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	2	2	✓
✓	13 13 1 2 1 12 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	3	3	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 int main() {
19     Graph *G;
20     int n, m;
21     if (scanf("%d %d", &n, &m) != 2) return 0;
22     G = (Graph *) malloc(sizeof(Graph));
23     init_graph(G, n);
24     for (int i = 0; i < m; i++) {
25         int u, v;
26         scanf("%d %d", &u, &v);
27         G->A[u][v] = 1;
28     }
29     return 0;
30 }

```

```

18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35
36 void DFS(Graph *pG, int u) {
37     //1. Đánh dấu u đã duyệt
38     //printf("Duyet %d\n", u); //Làm gì đó trên u
39     mark[u] = 1; //Đánh dấu nó đã duyệt
40
41     //2. Xét các đỉnh kề của u
42     for (int v = 1; v <= pG->n; v++)
43         if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
44             DFS(pG, v); //gọi đệ quy duyệt nó
45 }
46
47
48 int main() {
49     //1. Khai báo đồ thị G
50     Graph G;
51     //2. Đọc dữ liệu và dựng đồ thị
52     int n, m, u, v;
53     scanf("%d%d", &n, &m);
54     init_graph(&G, n);
55     for (int e = 0; e < m; e++) {
56         scanf("%d%d", &u, &v);
57         add_edge(&G, u, v);
58     }
59
60     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
61     for (int u = 1; u <= G.n; u++) {
62         mark[u] = 0;
63     }
64
65     //4. Duyệt toàn bộ đồ thị
66     int cnt = 0;
67     for (int u = 1; u <= G.n; u++)
68         if (mark[u] == 0) {
69             DFS(&G, u);
70             cnt++;
71         }
72
73     printf("%d\n", cnt);
74
75     return 0;
76 }
77
78

```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc một đồ thị **vô hướng** từ bàn phím và đếm số đỉnh của bộ phận liên thông của đỉnh 1.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u v mô tả cung (u, v) .

Đầu ra (Output)

- In ra số đỉnh của bộ phận liên thông của đỉnh 1.

Gợi ý

- Xem tài liệu thực hành.

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

For example:

Input	Result
4 3 2 1 1 3 2 4	4

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
```

```

31 //-----DFS-----
32
33 int cnt_edge;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     cnt_edge++;
37     visited[s] = 1;
38     // printf("%d \n", s);
39     for (int u = 1; u <= pG->n; u++) {
40         if (pG->A[s][u] > 0 && !visited[u]) {
41             DFS(pG, u);
42         }
43     }
44 }
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     cnt_edge = 0;
62     DFS(&G, 1);
63
64     // for (int i = 1; i <= G.n; i++) {
65     //     if (!visited[i]) {
66     //         cnt++;
67     //         DFS(&G, i);
68     //     }
69     // }
70     // }
71     printf("%d", cnt_edge);
72 }
73

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 2 4	4	4	✓
✓	4 2 1 2 3 4	2	2	✓
✓	4 2 1 4 2 3	2	2	✓

	Input	Expected	Got	
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5 6	13	13	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	10	10	✓
✓	13 13 1 2 1 12 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	5	5	✓

Passed all tests! ✓

Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 int main() {
19     Graph *G;
20     int n, m;
21     scanf("%d %d", &n, &m);
22     G = (Graph *) malloc(sizeof(Graph));
23     init_graph(G, n);
24     for (int i = 0; i < m; i++) {
25         int u, v;
26         scanf("%d %d", &u, &v);
27         G->A[u][v] = 1;
28     }
29     return 0;
30 }

```

```

18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35 int nb_u;
36
37 void DFS(Graph *pG, int u) {
38     //1. Đánh dấu u đã duyệt
39     //printf("Duyet %d\n", u); //Làm gì đó trên u
40     mark[u] = 1; //Đánh dấu nó đã duyệt
41     nb_u++;
42
43     //2. Xét các đỉnh kề của u
44     for (int v = 1; v <= pG->n; v++)
45         if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
46             DFS(pG, v); //gọi đệ quy duyệt nó
47 }
48
49
50 int main() {
51     //1. Khai báo đồ thị G
52     Graph G;
53     //2. Đọc dữ liệu và dựng đồ thị
54     int n, m, u, v;
55     scanf("%d%d", &n, &m);
56     init_graph(&G, n);
57     for (int e = 0; e < m; e++) {
58         scanf("%d%d", &u, &v);
59         add_edge(&G, u, v);
60     }
61
62     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
63     for (int u = 1; u <= G.n; u++) {
64         mark[u] = 0;
65     }
66
67     //4. Duyệt đồ thị từ đỉnh 1
68     nb_u = 0;
69     DFS(&G, 1);
70
71     printf("%d\n", nb_u);
72
73     return 0;
74 }
75
76

```

Correct

Marks for this submission: 1.00/1.00.

Viết chương trình đọc một đồ thị **vô hướng** từ bàn phím và đếm số đỉnh của bộ phận liên thông của đỉnh **s**. Đỉnh **s** được đọc từ bàn phím.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u và v mô tả cung (u, v) .
- Dòng cuối cùng chứa đỉnh s .

Đầu ra (Output)

- In ra số đỉnh của bộ phận liên thông của đỉnh s .

Gợi ý

- Xem tài liệu thực hành.

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

For example:

Input	Result
4 3 2 1 1 3 2 4 3	4

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
```

```

27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
31
32 //-----DFS-----
33 int cnt_edge;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     cnt_edge++;
37     visited[s] = 1;
38     // printf("%d \n", s);
39     for (int u = 1; u <= pG->n; u++) {
40         if (pG->A[s][u] > 0 && !visited[u]) {
41             DFS(pG, u);
42         }
43     }
44 }
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     cnt_edge = 0;
62     int s; scanf("%d",&s);
63     DFS(&G, s);
64
65     // for (int i = 1; i <= G.n; i++) {
66     //     if (!visited[i]) {
67     //         cnt++;
68     //         DFS(&G, i);
69     //     }
70     // }
71     // }
72     printf("%d", cnt_edge);
73 }
74

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 2 4 3	4	4	✓
✓	4 2 1 2 3 4 3	2	2	✓
✓	4 3 1 2 2 3 3 1 3	3	3	✓

	Input	Expected	Got	
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5 6 5	13	13	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 6	10	10	✓
✓	13 13 1 2 1 12 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5	3	3	✓
✓	13 14 1 2 1 12 3 7 3 12 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 2	10	10	✓

Question author's solution (C):

```

1 #include <stdio.h>
2
3 /* Khai báo CTDL Graph*/
4 #define MAX_N 100
5 typedef struct {
6     int n, m;
7     int A[MAX_N][MAX_N];
8 } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35 int nb_u;
36
37 void DFS(Graph *pG, int u) {
38     //1. Đánh dấu u đã duyệt
39     //printf("Duyet %d\n", u); //Làm gì đó trên u
40     mark[u] = 1; //Đánh dấu nó đã duyệt
41     nb_u++;
42
43     //2. Xét các đỉnh kề của u
44     for (int v = 1; v <= pG->n; v++)
45         if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
46             DFS(pG, v); //gọi đệ quy duyệt nó
47
48 }
49
50 int main() {
51     //1. Khai báo đồ thị G
52     Graph G;
53     //2. Đọc dữ liệu và dựng đồ thị
54     int n, m, u, v, s;
55     scanf("%d%d", &n, &m);
56     init_graph(&G, n);
57     for (int e = 0; e < m; e++) {
58         scanf("%d%d", &u, &v);
59         add_edge(&G, u, v);
60     }
61
62     scanf("%d", &s);
63     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
64     for (int u = 1; u <= G.n; u++) {
65         mark[u] = 0;
66     }
67
68     //4. Duyệt đồ thị từ đỉnh 1
69     nb_u = 0;
70     DFS(&G, s);
71
72     printf("%d\n", nb_u);
73
74     return 0;

```

74	return 0;
75	}
76	
77	

Correct

Marks for this submission: 1.00/1.00.



Viết chương trình đọc một đồ thị **vô hướng** từ bàn phím, tìm bộ phận liên thông có nhiều đỉnh nhất.

Đầu vào (Input)

Dữ liệu đầu vào được nhập từ bàn phím với định dạng:

- Dòng đầu tiên chứa 2 số nguyên n và m , tương ứng là số đỉnh và số cung.
- m dòng tiếp theo mỗi dòng chứa 2 số nguyên u và v mô tả cung (u, v) .

Đầu ra (Output)

- In ra số đỉnh của bộ phận liên thông có nhiều đỉnh nhất.

Gợi ý

- Xem tài liệu thực hành.

Hướng dẫn đọc dữ liệu và chạy thử chương trình

- Để chạy thử chương trình, bạn nên tạo một tập tin **dt.txt** chứa đồ thị cần kiểm tra.
- Thêm dòng **freopen("dt.txt", "r", stdin);** vào ngay sau hàm main(). Khi nộp bài, nhớ gỡ bỏ dòng này ra.
- Có thể sử dụng đoạn chương trình đọc dữ liệu mẫu sau đây:

```
freopen("dt.txt", "r", stdin); //Khi nộp bài nhớ bỏ dòng này.
Graph G;
int n, m, u, v, w, e;
scanf("%d%d", &n, &m);
init_graph(&G, n);

for (e = 0; e < m; e++) {
    scanf("%d%d", &u, &v);
    add_edge(&G, u, v);
}
```

For example:

Input	Result
4 3 2 1 1 3 2 4	4

Answer: (penalty regime: 10, 20, ... %)

```
1 #include <stdio.h>
2
3 #define MAX_SIZE 100
4 typedef int ElementType;
5
6 //-----Graph-----
7 typedef struct {
8     int n, m;
9     int A[MAX_SIZE][MAX_SIZE];
10 } Graph;
11
12 // init graph with n vertex
13 void init_graph(Graph* pG, ElementType n) {
14     pG->n = n;
15     pG->m = 0;
16
17     // cho toan bo ma tran ke bang 0
18     for (int i = 0; i <= pG->n; i++) {
19         for (int j = 0; j <= pG->n; j++) {
20             pG->A[i][j] = 0;
21         }
22     }
23 }
24
25 // add edge (u,v) to graph
26 void add_edge(Graph* pG, ElementType u, ElementType v) {
27     pG->A[u][v] = 1;
28     pG->A[v][u] = 1;
29     pG->m++;
30 }
```

```

31 //-----DFS-----
32
33 int max, cnt;
34 int visited[MAX_SIZE];
35 void DFS(Graph* pG, int s) {
36     cnt++;
37     visited[s] = 1;
38     // printf("%d \n", s);
39     for (int u = 1; u <= pG->n; u++) {
40         if (pG->A[s][u] > 0 && !visited[u]) {
41             DFS(pG, u);
42         }
43     }
44 }
45
46 int main() {
47     Graph G;
48     int n, m, u, v, e;
49     scanf("%d%d", &n, &m);
50     init_graph(&G, n);
51
52     for (e = 0; e < m; e++) {
53         scanf("%d%d", &u, &v);
54         add_edge(&G, u, v);
55     }
56
57     for (int i = 0; i <= G.n; i++) {
58         visited[i] = 0;
59     }
60
61     // DFS(&G, 1);
62
63     max = 0;
64     cnt = 0;
65     for (int i = 1; i <= G.n; i++) {
66         if (!visited[i]) {
67             cnt = 0;
68             DFS(&G, i);
69             max = cnt > max ? cnt : max;
70         }
71     }
72     printf("%d", max);
73 }
74

```

	Input	Expected	Got	
✓	4 3 2 1 1 3 2 4	4	4	✓
✓	4 2 1 2 3 4	2	2	✓
✓	4 3 1 2 2 3 3 1	3	3	✓

	Input	Expected	Got	
✓	13 16 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12 5 6	13	13	✓
✓	13 15 1 4 1 2 1 12 2 4 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	10	10	✓
✓	13 13 1 2 1 12 3 7 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	5	5	✓
✓	13 14 1 2 1 12 3 7 3 12 4 6 4 7 5 8 5 9 6 7 6 13 8 9 10 11 10 12 11 12	10	10	✓

Passed all tests! ✓



Question author's solution (C):

```

1  #include <stdio.h>
2
3  /* Khai báo CTDL Graph*/
4  #define MAX_N 100
5  typedef struct {
6      int n, m;
7      int A[MAX_N][MAX_N];
8  } Graph;
9
10 void init_graph(Graph *pG, int n) {
11     pG->n = n;
12     pG->m = 0;
13     for (int u = 1 ; u <= n; u++)
14         for (int v = 1 ; v <= n; v++)
15             pG->A[u][v] = 0;
16 }
17
18 void add_edge(Graph *pG, int u, int v) {
19     pG->A[u][v] += 1;
20     if (u != v)
21         pG->A[v][u] += 1;
22
23     pG->m++;
24 }
25
26 int adjacent(Graph *pG, int u, int v) {
27     return pG->A[u][v] > 0;
28 }
29
30
31
32
33 //Biến hỗ trợ dùng để lưu trạng thái của đỉnh: đã duyệt/chưa duyệt
34 int mark[MAX_N];
35 int nb_u;
36
37 void DFS(Graph *pG, int u) {
38     //1. Đánh dấu u đã duyệt
39     //printf("Duyet %d\n", u); //Làm gì đó trên u
40     mark[u] = 1; //Đánh dấu nó đã duyệt
41     nb_u++;
42
43     //2. Xét các đỉnh kề của u
44     for (int v = 1; v <= pG->n; v++)
45         if (adjacent(pG, u, v) && mark[v] == 0) //Nếu v chưa duyệt
46             DFS(pG, v); //gọi đệ quy duyệt nó
47 }
48
49
50 int main() {
51     //1. Khai báo đồ thị G
52     Graph G;
53     //2. Đọc dữ liệu và dựng đồ thị
54     int n, m, u, v, s;
55     scanf("%d%d", &n, &m);
56     init_graph(&G, n);
57     for (int e = 0; e < m; e++) {
58         scanf("%d%d", &u, &v);
59         add_edge(&G, u, v);
60     }
61
62     scanf("%d", &s);
63     //3. Khởi tạo mảng mark[u] = 0, với mọi u = 1, 2, ..., n
64     for (int u = 1; u <= G.n; u++) {
65         mark[u] = 0;
66     }
67
68     //4. Duyệt đồ thị từ đỉnh 1
69     int max_cnt = 0;
70     for (int u = 1; u <= G.n; u++)
71         if (mark[u] == 0) {
72             nb_u = 0;
73             DFS(&G, u);
74             if (nb_u > max_cnt)
75                 max_cnt = nb_u;
76         }
77
78     printf("%d\n", max_cnt);

```

```
79  
80     return 0;  
81 }  
82  
83
```

Correct

Marks for this submission: 1.00/1.00.

◀ * Bài tập 7. Kiểm tra đồ thị vô hướng liên thông

Jump to...

* Bài tập 9. Ứng dụng liên thông ▶

